Project Running Guide

This guide covers local environment setup, dependency installation, database configuration, execution commands, migration management, and containerization.

---

1. Prerequisites

Before setting up the project, make sure you have the following installed:

- **Node.js**: Version `20.x` or higher (Active LTS recommended).
- **NPM**: Installed automatically with Node.js (version 10.x+).
- **Git**: For code versioning and repository management.
- **PostgreSQL**: A running instance (local PG or cloud Neon PG).
- **Docker**: (Optional) For running via containers.

---

2. Project Installation

Clone the repository and install npm packages:

```
# Clone the repository
git clone https://github.com/Lokeessshhh/nestjs-booking-platform.git
cd nestjs-booking-platform

# Install dependencies
npm install
```

---

3. Environment Variables Configuration

Create a `.env` file in the project root based on the `.env.example` template:

```
cp .env.example .env
```

Open the `.env` file and configure the parameters:

```
PORT=3000
NODE_ENV=development

# Database Connection (Neon Postgres)
DATABASE_URL=postgresql://username:password@host:port/database?
sslmode=require

# JWT Secrets (Use strong secrets)
JWT_ACCESS_SECRET=your_jwt_access_secret_here
JWT_ACCESS_EXPIRATION=15m
JWT_REFRESH_SECRET=your_jwt_refresh_secret_here
JWT_REFRESH_EXPIRATION=7d
```

---

4. Database Setup & Migrations

Database synchronization (`synchronize: true`) is disabled. The schema must be updated using TypeORM migrations.
Migrations are **executed automatically on application startup** (`migrationsRun: true` in the DB config), but you can manage migrations manually:

```
# Build the application and run all pending migrations
npm run migration:run

# Revert the last applied migration
npm run migration:revert

# (Optional) Generate a new migration based on entity changes
npm run migration:generate
```

---

5. Running the Application

5.1 Local Execution

```
# Development mode (with hot-reload watching)
npm run start:dev

# Production build compilation
npm run build

# Run compiled production server
npm run start:prod
```

5.2 Accessing the API Docs (Swagger UI)

Once the server is running, you can access the Swagger API docs at:
👉 http://localhost:3000/api/docs

This interactive panel allows you to register users, authenticate, add header authorization tokens, and query all routes directly.

---

6. Running Tests

We have implemented unit testing for services and business logic. Run tests using:

```
# Execute Jest unit tests
npm run test

# Run tests in watch mode
npm run test:watch

# Generate test coverage report
npm run test:cov
```

---

7. Running with Docker

7.1 Dockerfile

The project is packaged with a multi-stage `Dockerfile` which keeps the production image extremely lightweight (only building compiled JS and prod dependencies):

```
# Build the Docker image
docker build -t nestjs-booking-platform .

# Run the Docker container
docker run -p 3000:3000 --env-file .env nestjs-booking-platform
```

7.2 Docker Compose

You can spin up the application along with a local PostgreSQL container using Docker Compose:

```
# Build and start services in detached mode
docker-compose up -d --build

# View logs
docker-compose logs -f

# Shut down services
docker-compose down
```

(By default, docker-compose.yml connects the app container to the Neon Postgres URL, but a local database container is also spun up on port 5432 for offline development compatibility)